NLP 243 Deep Learning for Natural Language Processing

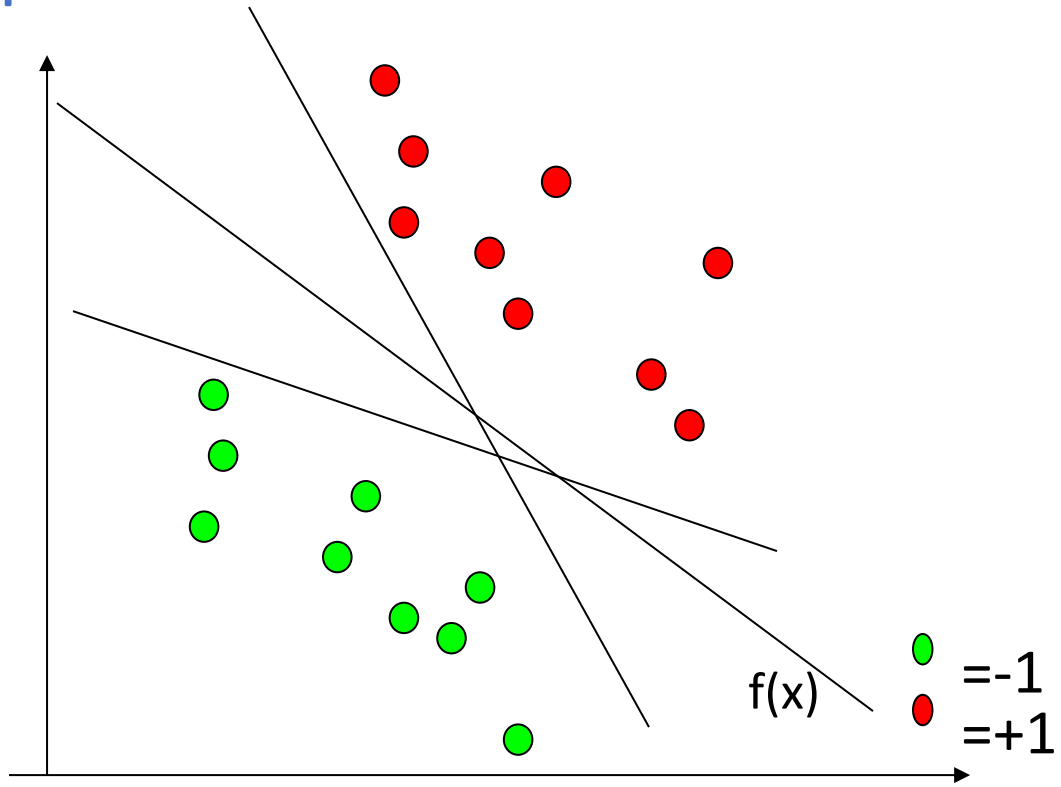
Fall 2023

SVMs, KNN and Linear Regression

Lecture Outline

- ML basics
- Training your own classifier: sklearn
- Naïve Bayes Classification
- Decision Trees
- Support Vector Machines
- K-nearest neighbors
- Linear Regression
- Course Projects

Support Vector Machines



Data: $<\mathbf{x}_i, y_i>$, $i=1,..,|D|$

$\mathbf{x}_i \in R^d$

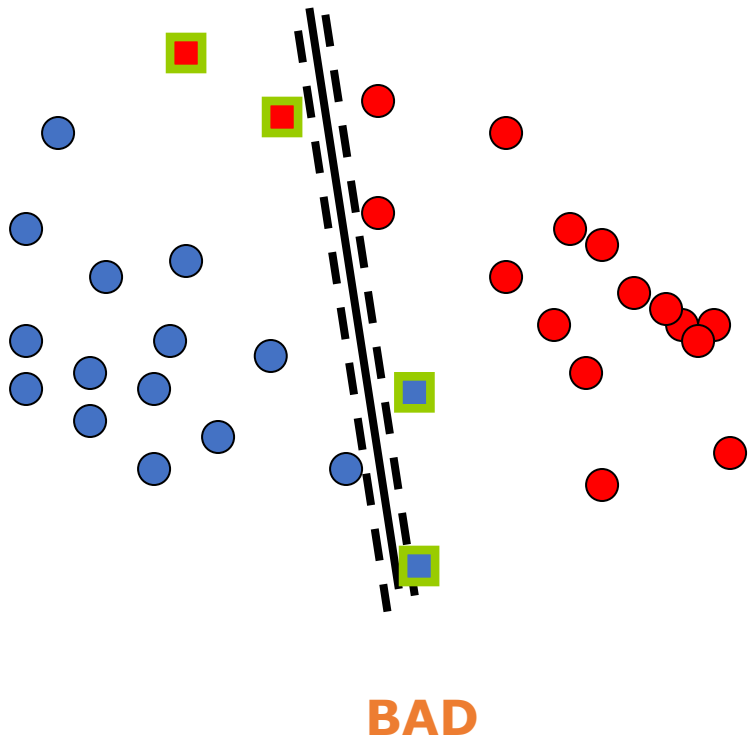
$y_i \in \{-1,+1\}$

All hyperplanes in R^d are parameterized by a vector (**w**) and a constant b (bias). Hence, they can be expressed as $\mathbf{w} \bullet \mathbf{x} + b = 0$ (remember the equation for a hyperplane from algebra!)

Our aim is to find such a hyperplane <u>$f(x)=\text{sign}(\mathbf{w} \bullet \mathbf{x}+b)$,</u> that can correctly classify our data.

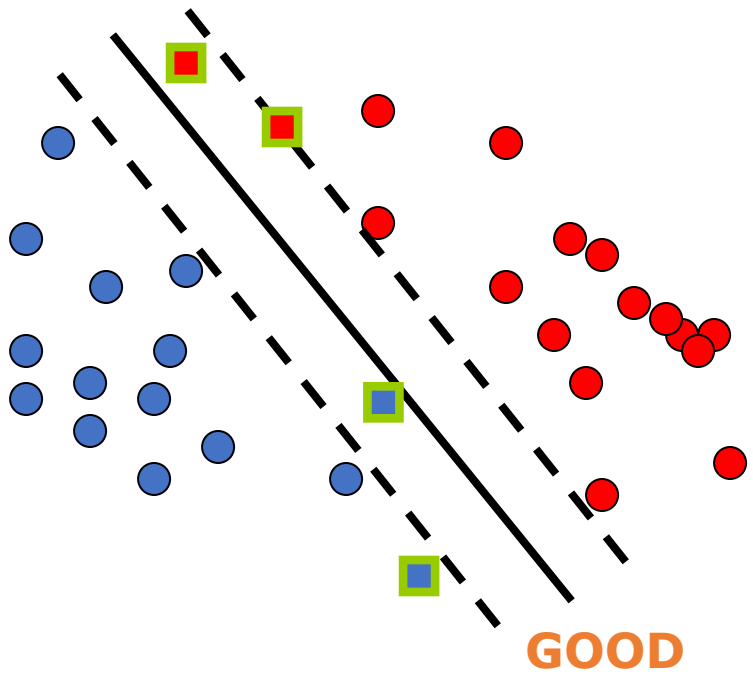
Support Vector Machines (SVM)

- **Another family of linear algorithms**
- **Intuition** (Vapnik, 1965)
- If the classes are linearly separable:
  - Separate the data
  - Place hyper-plane "far" from the data: **large margin**
  - Statistical results guarantee **good generalization**



SVM: Large margin classifier

- **Another family of linear algorithms**
- **Intuition** (Vapnik, 1965)
- If the classes are linearly separable:
  - Separate the data
  - Place hyper-plane "far" from the data: **large margin**
  - Statistical results guarantee **good generalization**

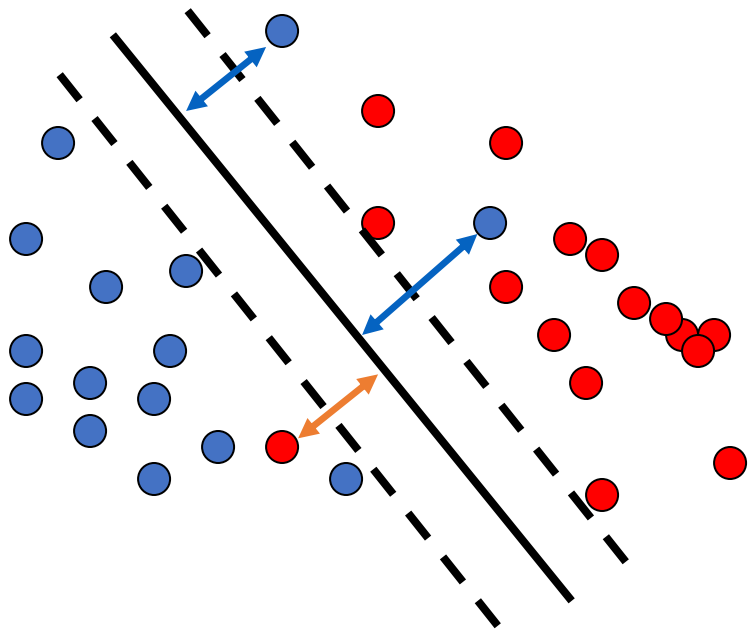


→ **Maximal Margin Classifier**

SVM: Large margin classifier

If **not linearly separable**

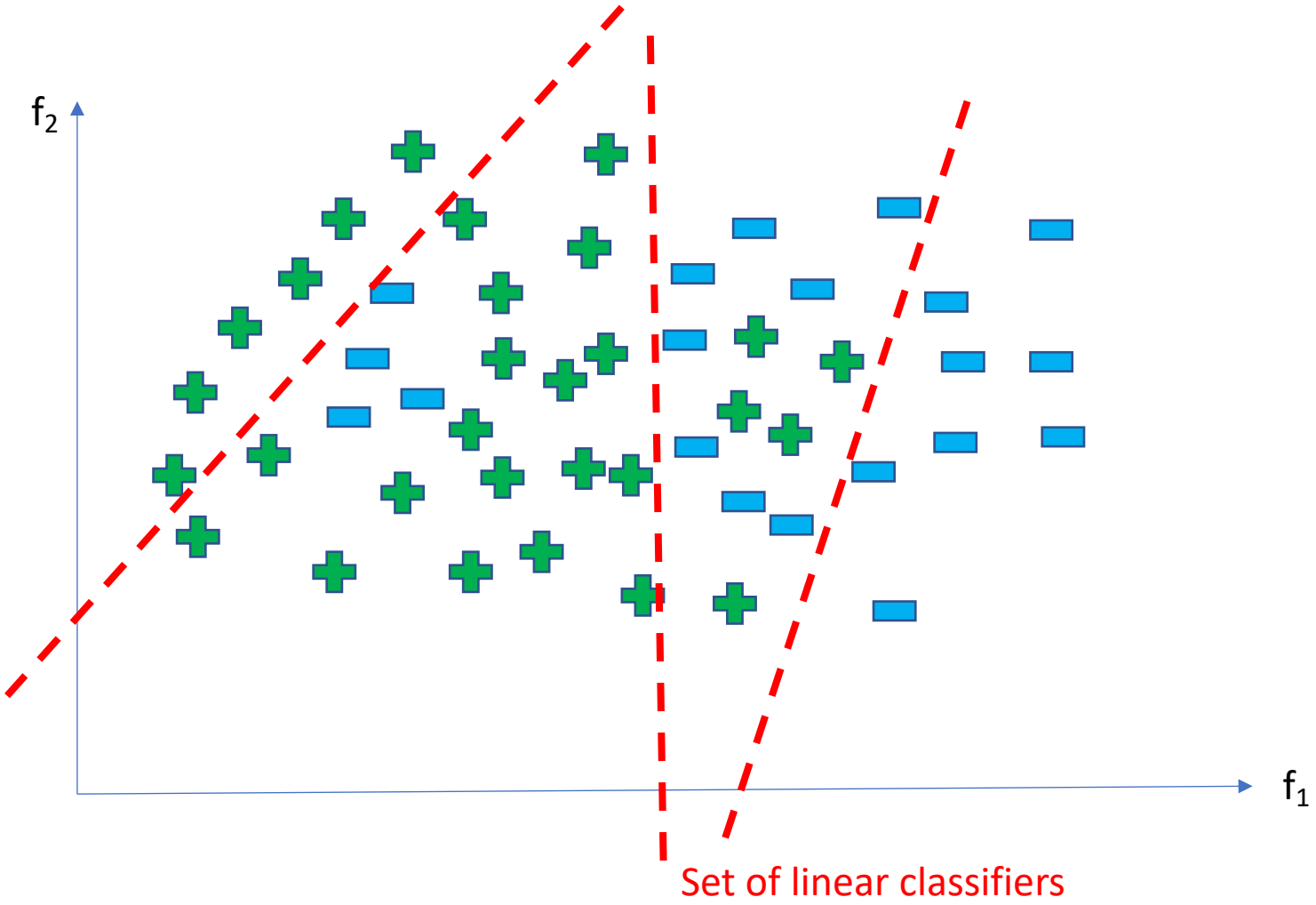
- **Allow** some **errors**
- Still, try to place hyperplane “far” from each class



SVM: Large Margin Classifiers

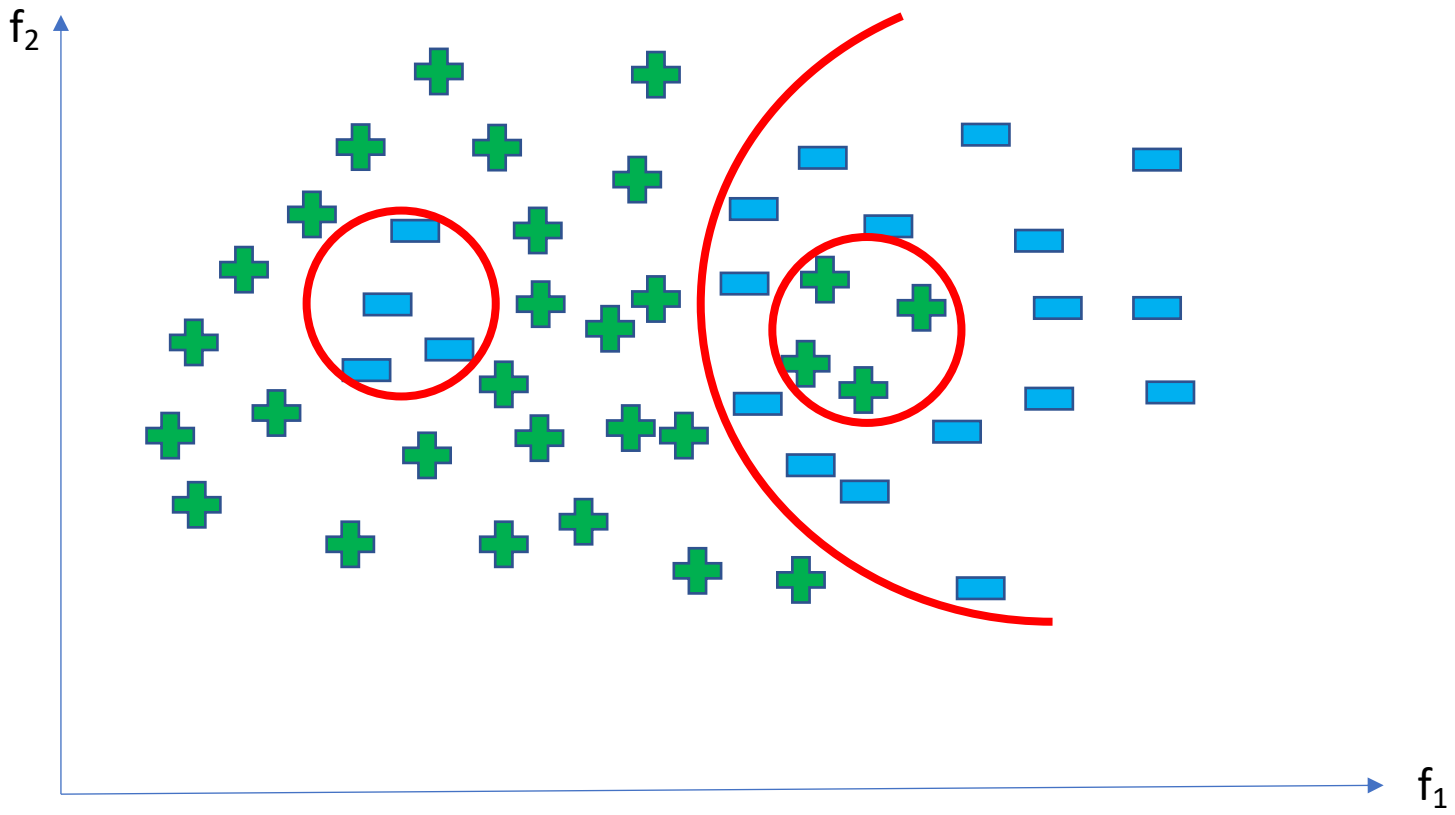
- Advantages
  - Theoretically better (better error bounds)
- Limitations
  - Computationally more expensive, large quadratic programming

So, what if it's a non-linear problem?



Two features: f_1 and f_2
Two classes: + and -
Depiction of all examples in the training set.

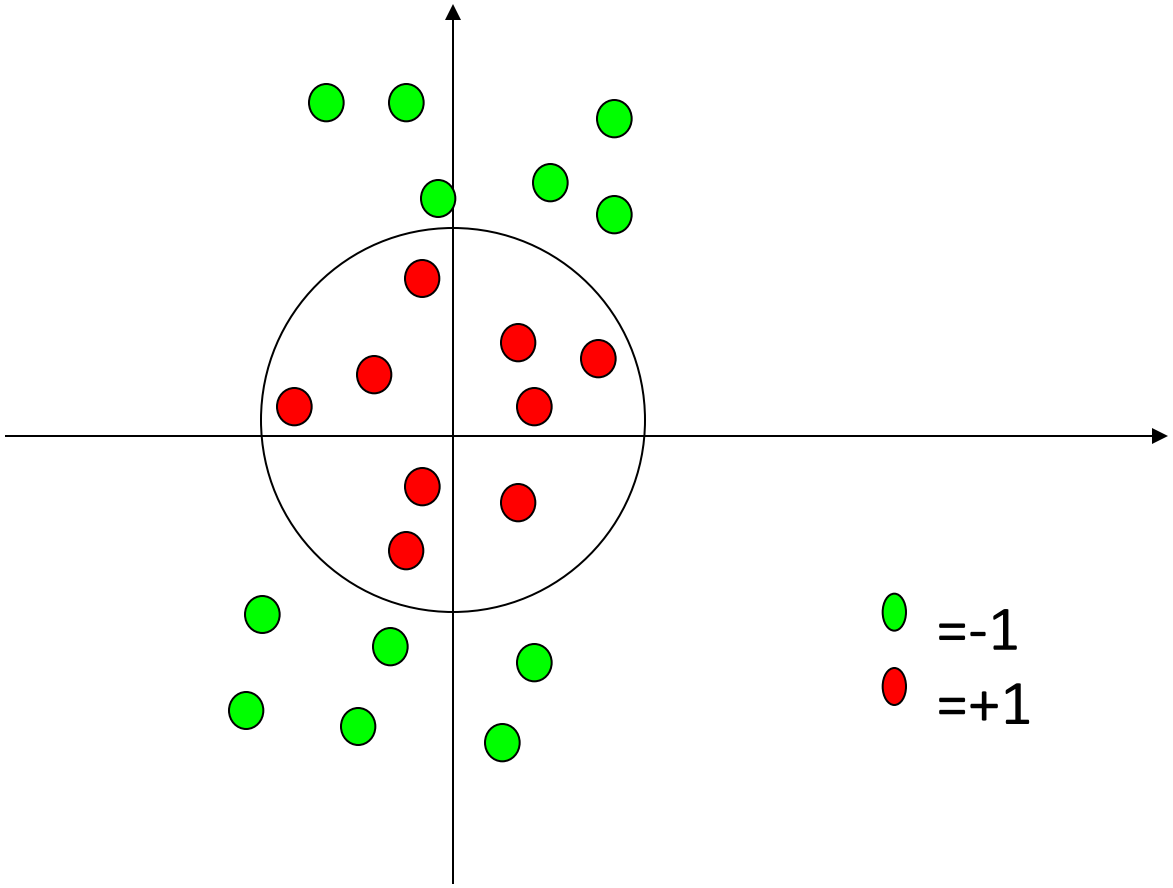
So, what if it's a non-linear problem?



Non-linear classifiers

Two features: f_1 and f_2
Two classes: + and -
Depiction of all examples in the training set.

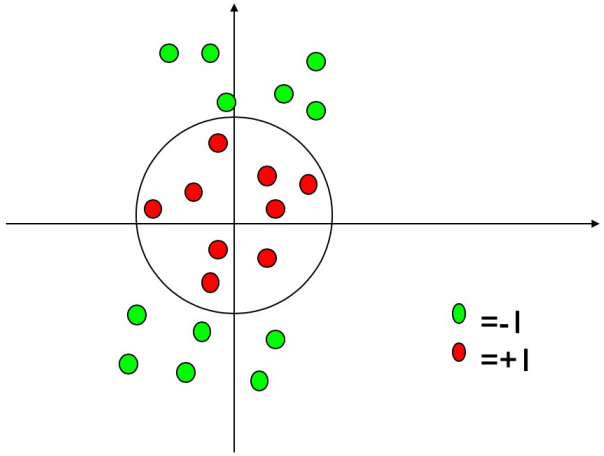
So, what if it’s a non-linear problem?



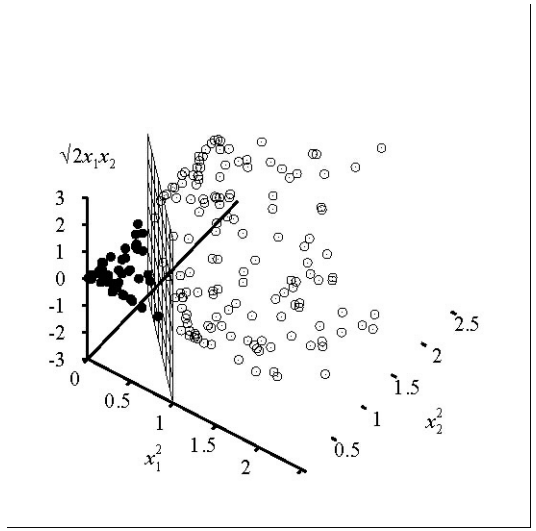
What if the decision function is not a linear?

Kernel Trick

General idea: The original feature space can always be mapped to some higher-dimensional feature space where the training set is separable.



$\Phi: \mathbf{x} \rightarrow \varphi(\mathbf{x})$



So, what if it's a non-linear problem?

- Kernel methods: A family of non-linear algorithms
- Transform the non-linear problem in a linear one (in a different feature space)
- Use linear algorithms to solve the linear problem in the new space
  - There are SVM learners in Scikit-Learn that have a kernel that does this kind of transform
  - http://scikit-learn.org/stable/modules/svm.html - svm-kernels
  - classifier = SklearnClassifier(svm.SVC()).train(train_sets)
  - class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', random_state=None)

From Gert Lanckriet, Statistical Learning Theory Tutorial

Lecture Outline

- ML basics
- Training your own classifier: sklearn
- Naïve Bayes Classification
- Decision Trees
- Support Vector Machines
- K-nearest neighbors
- Linear Regression
- Team Projects

Nearest Neighbor Methods

- Simplest
- Instance-based learning (also called memory-based learning)
  - Similar examples have the same label
  - We can classify an example with the label of a similar one.
- Image processing example:

→ cat

Example training set

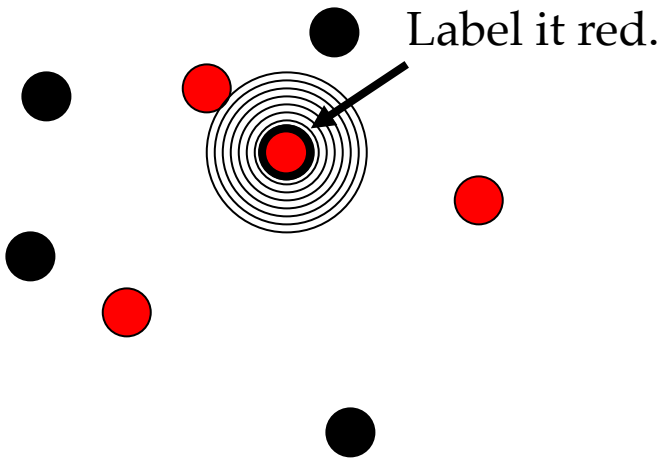
Images from CS231n by Fei-fei Li, Andrej Karpathy, and Justin Johnson

Nearest Neighbor Methods

- Algorithm:
  - Given a new example x for which we need to predict its label y
  - Find the most similar training examples
  - Classify x “like” these most similar examples
- **Lazy** learning algorithm
  - Processing of training examples postponed until inference time
  - Training ≈ gathering and storing examples

1-Nearest Neighbor

- Use only one neighbor
- Label an example with the label of the nearest example



Algorithm

- **Training:**

  for i = 1, ..., n in the training dataset:

  store training examples (x_i, y_i)

- **Inference:** label x_q

  closest point := None

  closest distance := ∞

  for i = 1, ..., n: # sequential search

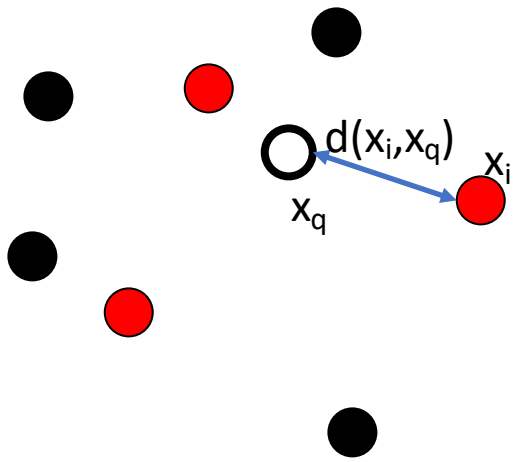
  current distance := $d(x_i, x_q)$

  if current distance < closest distance:

  closest distance := current distance

  closest point := x_i

  assign x_q is the label of closest point



What is the decision boundary?

- Voronoi (tessellation) diagram

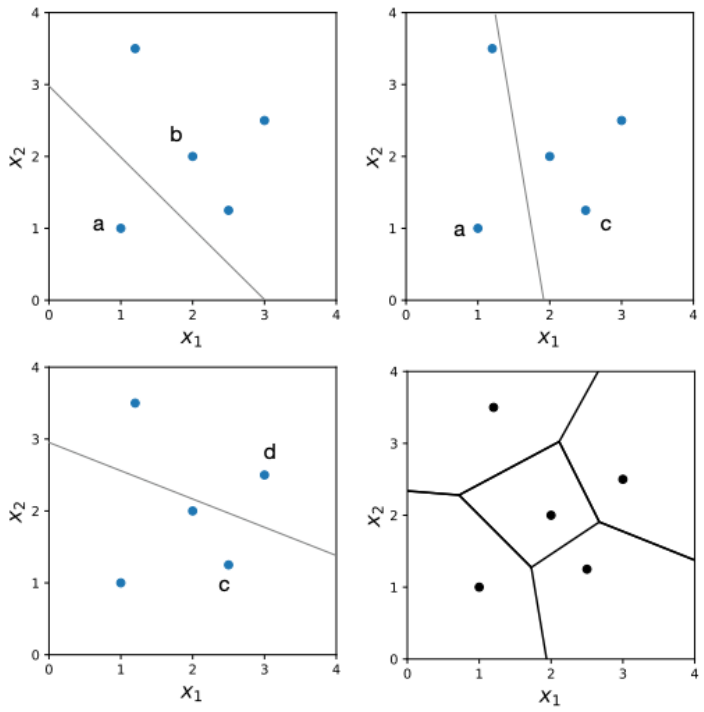


Figure 2: Illustration of the plane-partitioning of a two-dimensional dataset (features x_1 and x_2) via linear sigments between two training examples (a & b, a & c, and c & d) and the resulting Voronoi diagram (upper right corner)

Source: Wikipedia
A Voronoi tessellation emerges by radial growth from seeds outward.

What is the decision boundary?

- Voronoi (tessellation) diagram

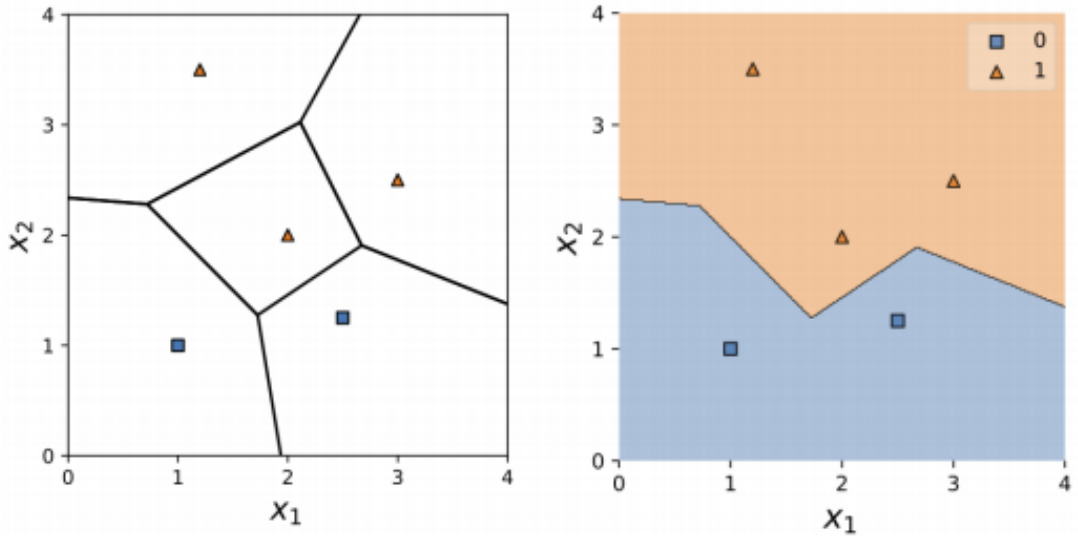
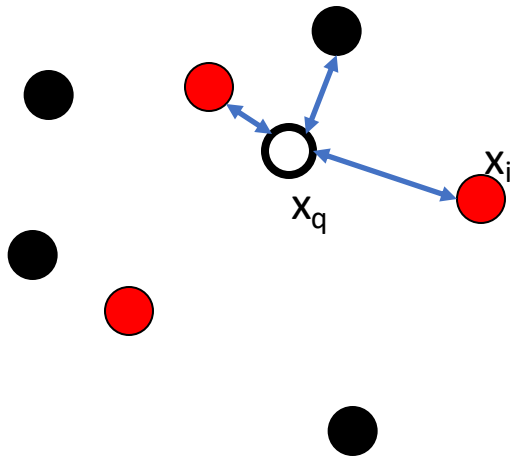


Figure 3: Illustration of the nearest neighbor decision boundary as the union of the polyhedra of training examples belonging to the same class.

k-Nearest Neighbors (kNN)

- Make the labeling decision based on k-nearest neighbors.
- Assign to a test sample the majority category label of its *k* nearest training samples
- In practice, *k* is usually chosen to be odd (for binary classification), so as to avoid ties
- For multi-class classification, k shouldn't be a multiple of the number of classes.



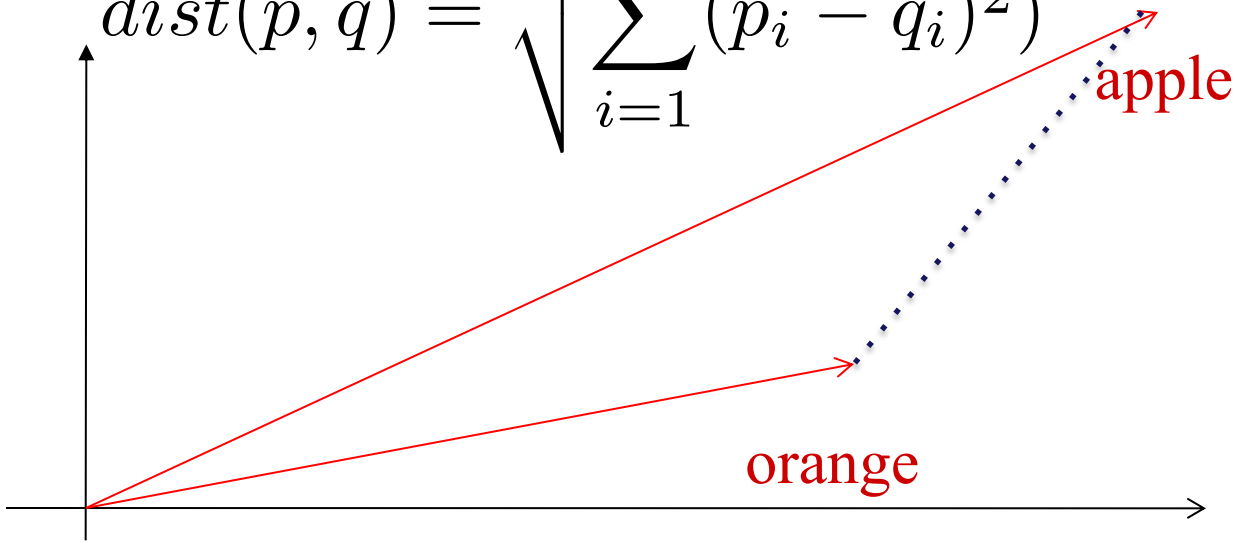
Questions

- How to represent examples?
- How to determine similarity?
- How many “similar training examples” to include?
- How to resolve inconsistencies between the training examples?

Euclidean (L2) Distance

- a dissimilarity measure:

$$dist(\vec{p}, \vec{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2)}$$



Distance Metrics

Minkowsky:

$$D(\boldsymbol{x},\boldsymbol{y}) = \left(\sum_{i=1}^{m} |x_i - y_i|^r\right)^{1/r}$$

Euclidean:

$$D(\boldsymbol{x},\boldsymbol{y}) = \sqrt{\sum_{i=1}^{m} (x_i - y_i)^2}$$

Manhattan / city-block:

$$D(\boldsymbol{x},\boldsymbol{y}) = \sum_{i=1}^{m} |x_i - y_i|$$

Camberra:

$$D(\boldsymbol{x},\boldsymbol{y}) = \sum_{i=1}^{m} \frac{|x_i - y_i|}{|x_i + y_i|}$$

Chebychev:

$$D(\boldsymbol{x},\boldsymbol{y}) = \max_{i=1}^{m} |x_i - y_i|$$

Quadratic:

$$D(\boldsymbol{x},\boldsymbol{y}) = (\boldsymbol{x} - \boldsymbol{y})^T Q(\boldsymbol{x} - \boldsymbol{y}) = \sum_{j=1}^{m}\left(\sum_{i=1}^{m} (x_i - y_i) q_{ji}\right)(x_j - y_j)$$

Q is a problem-specific positive definite $m \times m$ weight matrix

Mahalanobis:

$$D(\boldsymbol{x},\boldsymbol{y}) = [\det V]^{1/m} (\boldsymbol{x} - \boldsymbol{y})^{\mathrm{T}} V^{-1} (\boldsymbol{x} - \boldsymbol{y})$$

V is the covariance matrix of $A_1..A_m$, and A_j is the vector of values for attribute j occuring in the training set instances $1..n$.

Correlation:

$$D(\boldsymbol{x},\boldsymbol{y}) = \frac{\sum_{i=1}^{m} (x_i - \overline{x_i})(y_i - \overline{y_i})}{\sqrt{\sum_{i=1}^{m} (x_i - \overline{x_i})^2 \sum_{i=1}^{m} (y_i - \overline{y_i})^2}}$$

$\overline{x_i} = \overline{y_i}$ and is the average value for attribute i occuring in the training set.

Chi-square:

$$D(\boldsymbol{x},\boldsymbol{y}) = \sum_{i=1}^{m} \frac{1}{sum_i}\left(\frac{x_i}{size_{\boldsymbol{x}}} - \frac{y_i}{size_{\boldsymbol{y}}}\right)^2$$

sum_i is the sum of all values for attribute i occuring in the training set, and $size_{\boldsymbol{x}}$ is the sum of all values in the vector $\boldsymbol{x}$.

Kendall's Rank Correlation:

$$D(\boldsymbol{x},\boldsymbol{y}) = 1 - \frac{2}{n(n-1)} \sum_{i=1}^{m} \sum_{j=1}^{i-1} \mathrm{sign}(x_i - x_j)\,\mathrm{sign}(y_i - y_j)$$

sign(x)=-1, 0 or 1 if $x < 0$, $x = 0$, or $x > 0$, respectively.

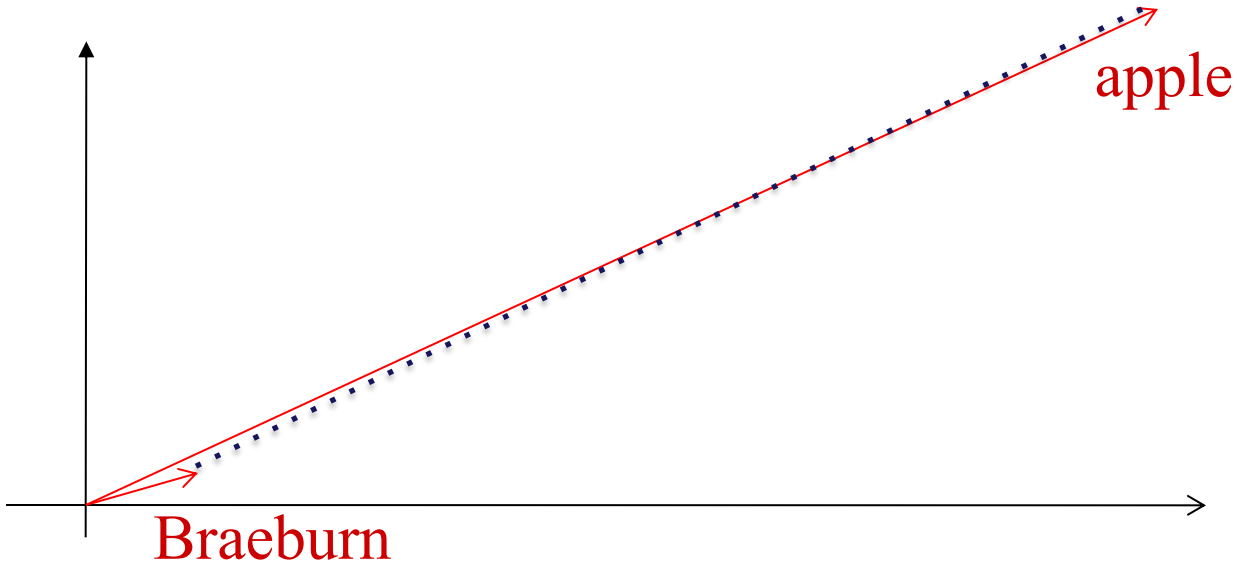
Figure 1. Equations of selected distance functions. ($\boldsymbol{x}$ and $\boldsymbol{y}$ are vectors of m attribute values).

Possible Issues:

- Most distance measures were designed for linear/real-valued attributes
- How to best handle discrete/nominal values?

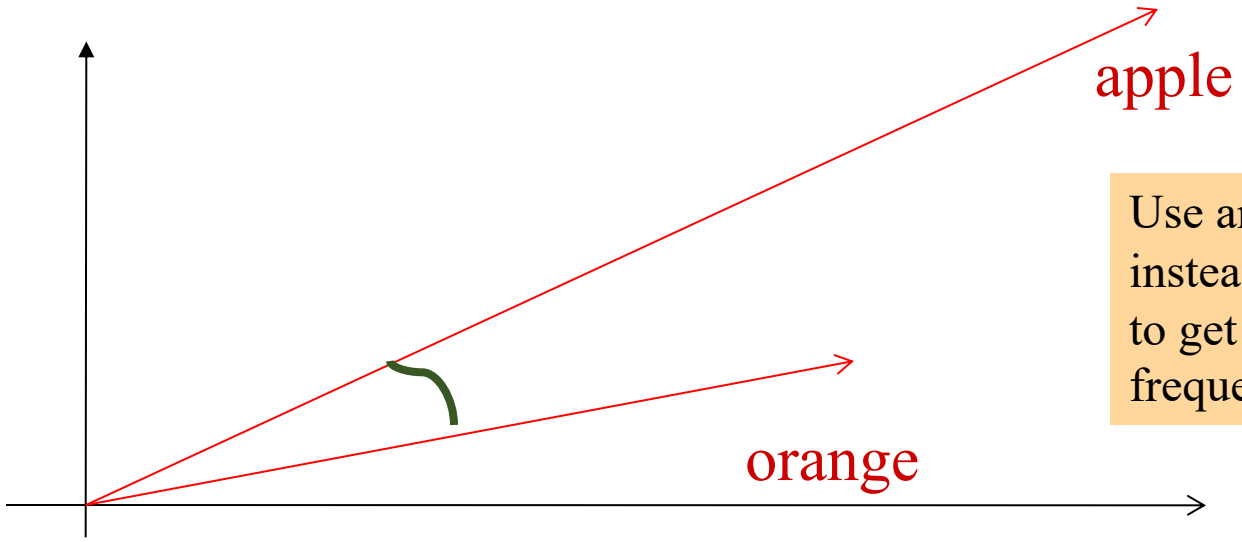
Euclidean (L2) Distance

- Problem: If features are context word counts, very sensitive to word frequency!



Cosine Similarity

$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$



Use angle between vectors instead of point distance to get around word frequency issues

Cosine for computing similarity

$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

	large	data	computer
apricot	1	0	0
digital	0	1	2
information	1	6	1

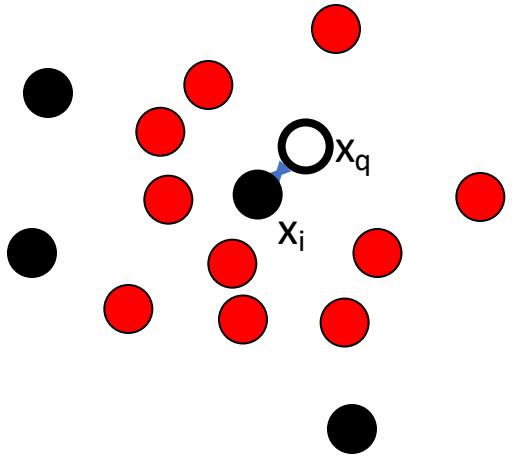
Which pair of words is more similar?

cosine(apricot,information) = $\frac{1+0+0}{\sqrt{1+0+0}\ \sqrt{1+36+1}} = \frac{1}{\sqrt{38}} = .16$

cosine(digital,information) = $\frac{0+6+2}{\sqrt{0+1+4}\ \sqrt{1+36+1}} = \frac{8}{\sqrt{38}\sqrt{5}} = .58$

cosine(apricot,digital) = $\frac{0+0+0}{\sqrt{1+0+0}\ \sqrt{0+1+4}} = 0$

Deciding on k



- Large k:
  - Makes KNN less sensitive to noise
- Small k:
  - Allows capturing finer structure of space

Pick k not too large, but not too small (depends on data)

Curse of dimensionality

- Prediction accuracy can quickly degrade when number of attributes grows.
  - Irrelevant attributes easily “swamp” information from relevant attributes
  - When many irrelevant attributes, similarity/distance measure becomes less reliable

- Remedy
  - Try to remove irrelevant attributes in pre-processing step
  - Weight attributes differently
  - Increase k (but not too much)

kNN Pros and Cons

- **Storage:** all training examples are stored in memory
  - A decision tree or linear classifier is much smaller
- **Time:** to classify *x*, you need to loop over <u>all</u> training examples *(x',y')* to compute distance between *x* and *x'*.
  - You get predictions for every class *y*
    - kNN is nice when there are many many classes
  - Actually, there are some tricks to speed this up…especially when data is sparse (e.g., text)
- Need distance/similarity measure and attributes that "match" target function.

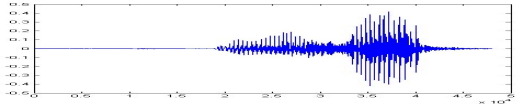
Lecture Outline

- ML basics
- Training your own classifier: sklearn
- Naïve Bayes Classification
- Decision Trees
- Support Vector Machines
- K-nearest neighbors
- Linear Regression
- Course Projects

Slide from Lecture 1

Learning ≈ Looking for a Function

- Speech Recognition

$f($  $) =$ "the book"

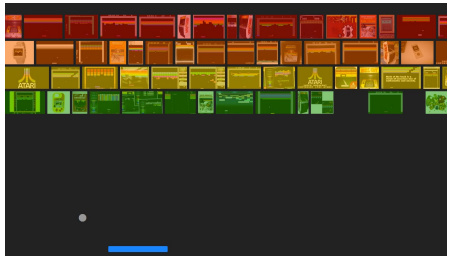
- Handwriting Recognition

$f($ $) =$ "2"

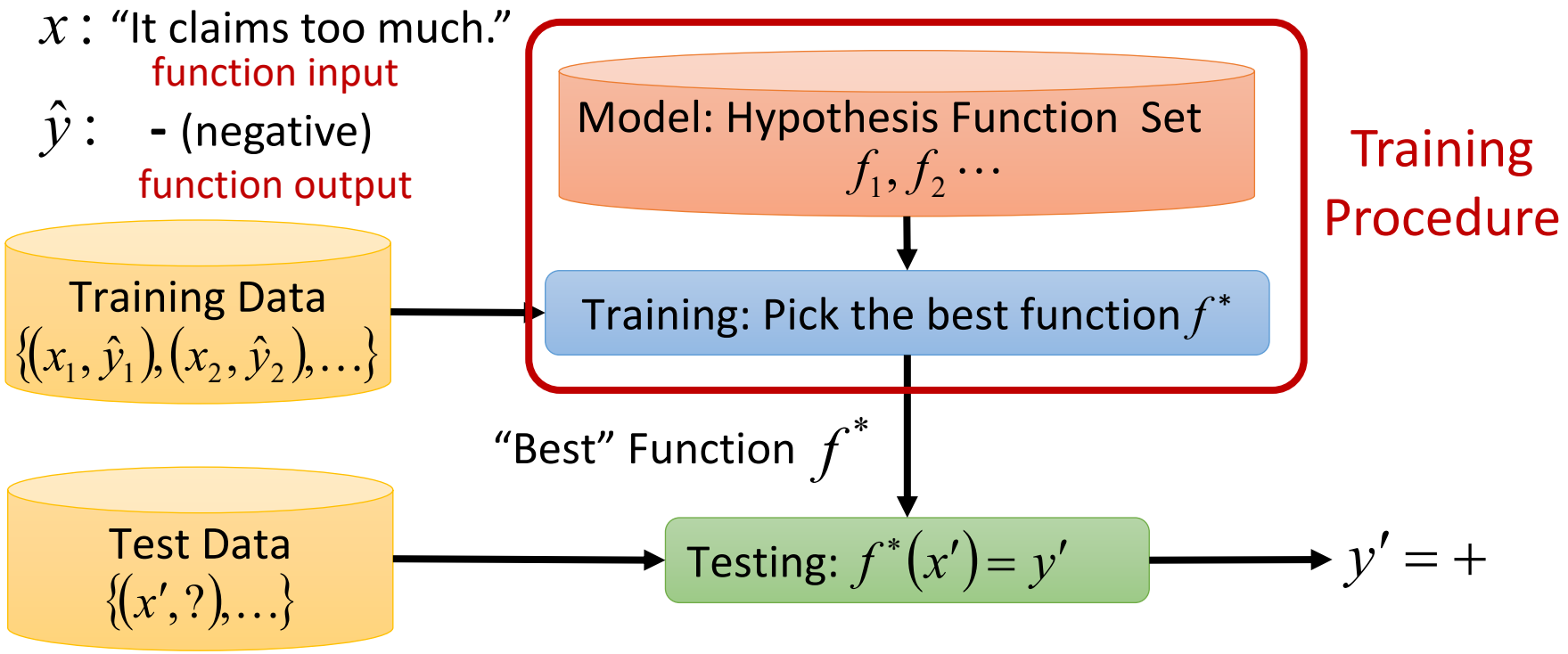
- Weather forecast

$f($ Thursday $) =$ " Saturday"

- Play video games

$f($  $) =$ "move left"

Machine Learning Framework



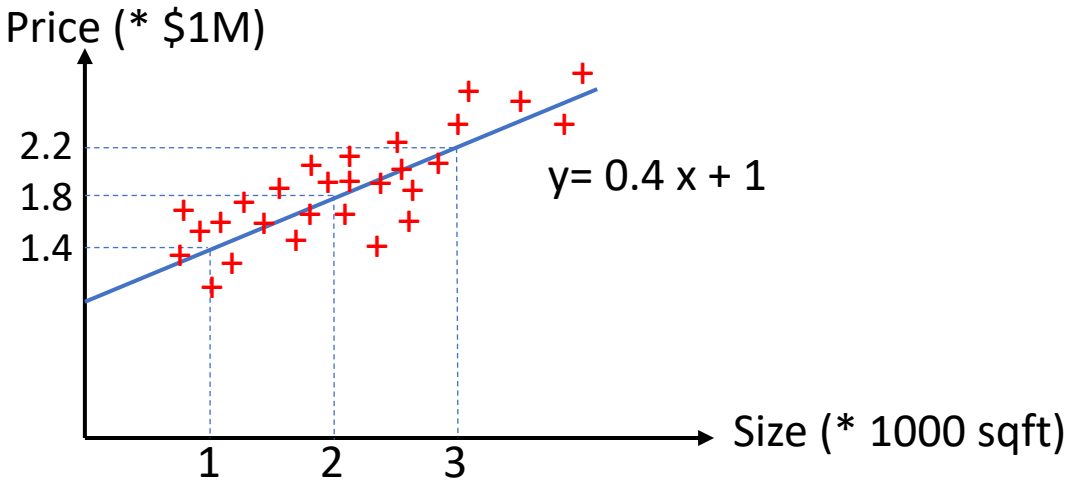
Training aims to pick the best function given the observed examples
Testing predicts the label using the learned function

Linear Regression

- **Regression**: modeling the relationship between data points **x** and corresponding real-valued targets y
  - Example: Predicting prices of homes
  - Label: Price, $y^{(i)}$
  - Features: Size, Age
    - $x^{(i)}=[x^{(i)}_1, x^{(i)}_2]$

2019 Sales:

Size (sqft)	Price
2100	$1.90M
1600	$1.56M
3200	$2.40M
…	…



Linear Regression (cont.)

- Assumption:
  - the relationship between the *features* $\mathbf{x}$ and targets y is linear,
  - i.e., y can be expressed as a weighted sum of the inputs $\mathbf{x}$, give or take some noise on the observations

$$\text{price} = w_1 . x_1 + w_2 . x_1 + b$$

weights

Bias
(offset or intercept)

Linear Regression: Modeling bivariate data

- Bivariate data: $(x_1,y_1), (x_2, y_2), \ldots, (x_n,y_n)$
- Model: $y_i = f(x_i) + e_i$ Random error

- Supervised approach!
- Model allows us to predict the value of y for any given value of x.
  - x is called the independent or predictor variable.
  - y is the dependent or response variable.

Examples of $f(x)$

- lines: $f(x) = mx + b$
- polynomials: $f(x) = ax^2 + bx + c$
- others: $f(x) = a/x + b$

  $f(x) = a\ sin(x) + b$

  $f(x) = a\ sqrt(x)$

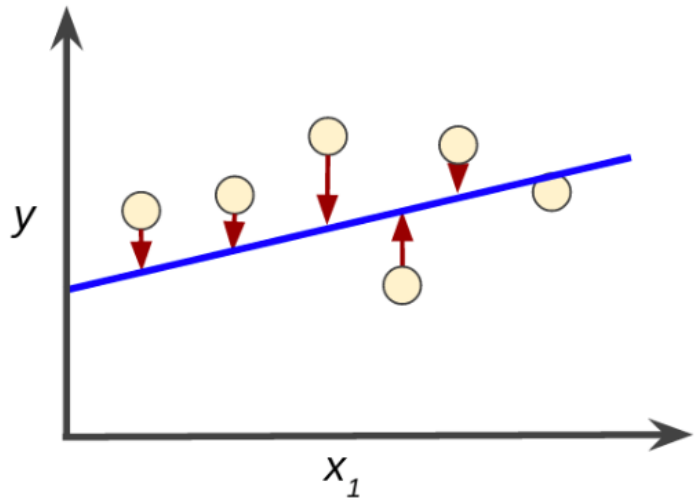
Assumptions for Linear Regression

- The relationship between *x* and *y* is linear
- *y* is distributed normally at each value of *x*, and the variance of *y* at every value of *x* is the same
- The observations are independent

Mean Squared Error

- y_i is the actual value, $f(x_i)$ is the prediction, and their difference is the error.
- Commonly used function, (mean) squared error:

$$MS = \sum_{i=1}^{n} (y_i - f(x_i))^2$$



Case Study: $f(x) = mx + b$

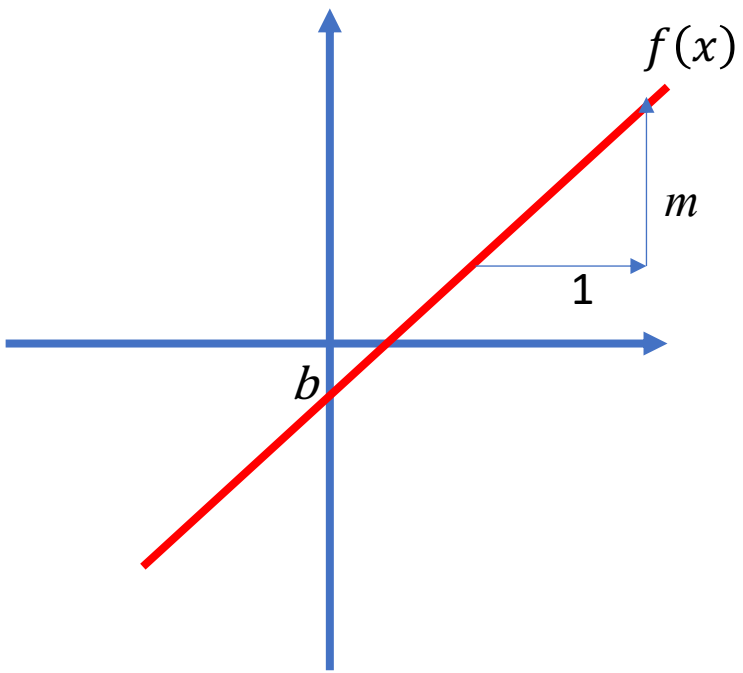
- (Mean) squared error:

$$MS(m, b) = \sum_{i=1}^{n} (y_i - mx_i - b)^2$$

- To estimate the values that minimize the mean squared error, we need to take the derivative with respect to the two variables m and b:

$$\frac{\partial MS(m, b)}{\partial b} = 0 \quad \text{and} \quad \frac{\partial MS(m, b)}{\partial m} = 0$$

Case Study: $f(x) = mx + b$



- b is called bias.
- m is the slope of $f(x)$
- Every time x moves by 1, y moves by m

Case Study: $f(x) = mx + b$

- We need to solve these two equations to find m and b, to be able to find the function $f(x_i)$.
- Beginning with the first one, for b:

$$\frac{\partial SS(m,b)}{\partial b} = \frac{\partial \sum_{i=1}^{n}(y_i - mx_i - b)^2}{\partial b} = \sum_{i=1}^{n} 2\,(y_i - mx_i - b)(-1) = 0$$

$$\sum_{i=1}^{n} y_i - m\sum_{i=1}^{n} x_i - nb = 0$$

$$b = \frac{1}{n}\sum_{i=1}^{n} y_i - m\,\frac{1}{n}\sum_{i=1}^{n} x_i$$

Case Study: $f(x) = mx + b$

- We can use $\bar{y}$ and $\bar{x}$ to represent the mean values of the x and y coordinates, that is all the x and y values in our data:

$$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i \qquad \text{and} \qquad \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Then we can simply re-write the equation for b:

$$b = \frac{1}{n}\sum_{i=1}^{n} y_i - m\,\frac{1}{n}\sum_{i=1}^{n} x_i$$

$$b = \bar{y} - m\,\bar{x}$$

Case Study: $f(x) = mx + b$

- Now, let's find the value of m:

$$\frac{\partial SS(m,b)}{\partial m} = \frac{\partial \sum_{i=1}^{n}(y_i - mx_i - b)^2}{\partial m} = \frac{\partial \sum_{i=1}^{n}(y_i - mx_i - \bar{y} + m\,\bar{x})^2}{\partial m} = 0$$

$$\sum_{i=1}^{n} 2\,(y_i - mx_i - \bar{y} + m\,\bar{x})\,(-x_i + \bar{x}) = 0$$

$$\sum_{i=1}^{n} -y_i x_i + y_i\,\bar{x} + m\,x_i^2 - m\,x_i\,\bar{x} + \bar{y}\,x_i - \bar{y}\,\bar{x} - m\,\bar{x}\,x_i + m\,\bar{x}^2 = 0$$

Case Study: $f(x) = mx + b$

$$\sum_{i=1}^{n} -y_i x_i + y_i\,\bar{x} + m\,x_i^2 - m\,x_i\,\bar{x} + \bar{y}\,x_i - \bar{y}\,\bar{x} - m\,\bar{x}\,x_i + m\,\bar{x}^2 = 0$$

$$\sum_{i=1}^{n} -y_i x_i + y_i\,\bar{x} + \bar{y}\,x_i - \bar{y}\,\bar{x} + m\,x_i^2 - m\,x_i\,\bar{x} - m\,\bar{x}\,x_i + m\,\bar{x}^2 = 0$$

$$\sum_{i=1}^{n} -y_i x_i + y_i\,\bar{x} + \bar{y}\,x_i - \bar{y}\,\bar{x} + m\,x_i^2 - 2m\,x_i\,\bar{x} + m\,\bar{x}^2 = 0$$

Case Study: $f(x) = mx + b$

$$\sum_{i=1}^{n} \underbrace{-y_i x_i + y_i\,\bar{x} + \bar{y}\,x_i - \bar{y}\,\bar{x}} + \underbrace{m\,x_i^2 - 2m\,x_i\,\bar{x} + m\,\bar{x}^2} = 0$$

$$\sum_{i=1}^{n} (\bar{y} - y_i)(x_i - \bar{x}) + m\,(x_i - \bar{x})^2 = 0$$

Therefore,

$$m = \frac{\sum_{i=1}^{n} (\bar{y} - y_i)(\bar{x} - x_i)}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

Case Study: $f(x) = mx + b$

Summary:

$$b = \bar{y} - m\,\bar{x} \qquad\qquad m = \frac{\sum_{i=1}^{n}(\bar{y} - y_i)(\bar{x} - x_i)}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Lecture Outline

- ML basics
- Training your own classifier: sklearn
- Naïve Bayes Classification
- Decision Trees
- Support Vector Machines
- K-nearest neighbors
- Linear Regression
- Course Projects

Class Project: Apply Existing NNets to Tasks

1. Define Task:
   - Summarization
   - Named Entity Recognition
   - Sentence Similarity

2. Define Dataset
   - Search for academic datasets
   - They already have baselines
   - E.g: Document Understanding Conference (DUC) for Summarization

Class Project: Apply Existing NNets to Tasks

3. Define your metric
   - Search online for well established metrics on this task
   - Summarization: Rouge (Recall-Oriented Understudy for Gisting Evaluation) which defines n-gram overlap to human summaries

4. Split your dataset!
   - Train/Dev/TesT
   - Academic dataset often come pre-split
   - Don’t look at the test split until ~1 week before deadline! (or at most once a week)

Class Project: Apply Existing NNets to Tasks

5. Establish a baseline
   - Implement the simplest model (often logistic regression on unigrams and bigrams) first
   - Compute metrics on train AND dev
     - Analyze errors
     - If metrics are amazing and no errors: done, problem was too easy, restart :)

6. Implement existing neural net model
   - Compute metric on train and dev
   - Analyze output and errors
   - Minimum bar for this class

Class Project: Apply Existing NNets to Tasks

7. Visualize the dataset
   - Collect summary statistics
   - Look at errors
   - Analyze how different hyperparameters affect performance

8. Try out different model variants
   - Word vector averaging model (neural bag of words)
   - Convolutional neural network
   - Recurrent neural network
   - Transformer Networks

https://semeval.github.io/SemEval2024/tasks.html

Semantic Relations

- **Task 1: Semantic Textual Relatedness for African and Asian Languages** ([contact organizers], [join task mailing list])
  Nedjma OUSIDHOUM, Shamsuddeen Hassan Muhammad, Mohamed Abdalla, Krishnapriya Vishnubhotla, Vladimir Araujo, Meriem Beloucif, Idris Abdulmumin, Seid Muhie Yimam, Thamar Solorio, Monojit Choudhury, Saif M. Mohammad

- **Task 2: Safe Biomedical Natural Language Inference for Clinical Trials** ([contact organizers], [join task mailing list])
  Mael Jullien, Marco Valentino, Andre Freitas

https://semeval.github.io/SemEval2024/tasks.html

Discourse and Argumentation

- **Task 3: The Competition of Multimodal Emotion Cause Analysis in Conversations** ([contact organizers], [join task mailing list])
  Rui Xia, Jianfei Yu, Fanfan Wang, Erik Cambria
- **Task 4: Multilingual Detection of Persuasion Techniques in Memes** ([contact organizers], [join task mailing list])
  Dimitar Iliyanov Dimitrov, Giovanni Da San Martino, Fabrizio Silvestri, Preslav Nakov, Firoj Alam
- **Task 5: Argument Reasoning in Civil Procedure** ([contact organizers], [join task mailing list])
  Lena Held, Ivan Habernal

https://semeval.github.io/SemEval2024/tasks.html

LLM Capabilities

- **Task 6: SHROOM, a Shared-task on Hallucinations and Related Observable Overgeneration Mistakes** ([contact organizers], [join task mailing list])
  Elaine Zosa, Raúl Vázquez, Jörg Tiedemann, Vincent Segonne, Alessandro Raganato, Timothee Mickus, Marianna Apidianaki

- **Task 7: NumEval: Numeral-Aware Language Understanding and Generation** ([contact organizers], [join task mailing list])
  Chung-Chi Chen, Hen-Hsen Huang, Hiroya Takamura, Hsin-Hsi Chen

- **Task 8: Multidomain, Multimodal and Multilingual Machine-Generated Text Detection** ([contact organizers], [join task mailing list])
  Yuxia Wang, Alham Fikri Aji, Artem Shelmanov, Chenxi Whitehouse, Petar Ivanov, Jonibek Mansurov, Jinyan Su, Tarek Mahmoud, Osama Mohammed Afzal, Preslav Nakov

https://semeval.github.io/SemEval2024/tasks.html

Knowledge Representation and Reasoning

- **Task 9: BRAINTEASER: A Novel Task Defying Common Sense** ([contact organizers], [join task mailing list])
  Yifan Jiang, Filip Ilievski, Kaixin Ma

- **Task 10: Emotion Discovery and Reasoning its Flip in Conversation** ([contact organizers], [join task mailing list])
  Shivani Kumar, Md Shad Akhtar, Tanmoy Chakraborty, Erik Cambria

Task 1: Semantic Textual Relatedness (STR)

Motivation

The semantic relatedness of two language units has long been considered fundamental to understanding meaning (*Halliday and Hassan 1976, Miller and Charles 1991*), and automatically determining relatedness has many applications such as evaluating sentence representation methods, question answering, and summarization (*Abdalla et al. 2023*).

Two sentences are considered semantically similar when they have a paraphrasal or entailment relation. On the other hand, relatedness is a much broader concept that accounts for all the commonalities between two sentences: whether they are on the same topic, express the same view, originate from the same time period, one elaborates on (or follows from) the other, etc. For instance, for the following sentence pairs:

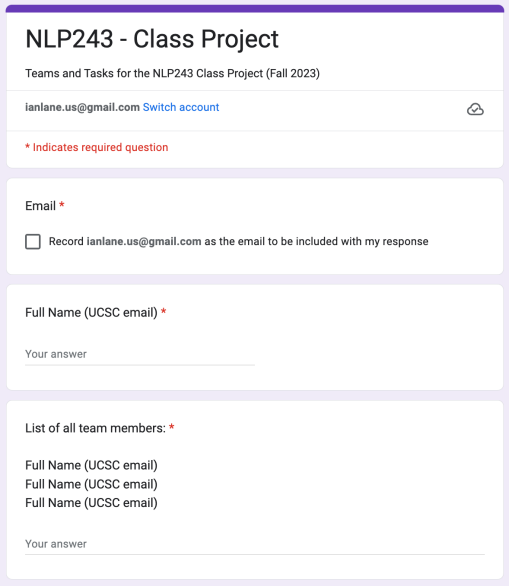
PAIRS	SENTENCE 1	SENTENCE 2
1	There was a lemon tree next to the house.	The boy enjoyed reading under the lemon tree.
2	There was a lemon tree next to the house.	The boy was an excellent football player.

Most people will agree that the sentences in pair 1 are more related than the sentences in pair 2.

Much of past NLP work is on semantic similarity, and largely focused on English. In this shared task, we cover the following languages: Afrikaans, Algerian Arabic, Amharic, English, Hausa, Hindi, Indonesian, Kinyarwanda, Marathi, Morrocan Arabic, Modern Standard Arabic, Punjabi, Spanish, and Telugu.

https://forms.gle/yTVyWc89ZFaTbzkG6

- Finalize Teams (**Friday, October 20th @ 12:00 noon**)
  - Teams of 3
  - Teams of 2 or 4 allowable with written permission (email)
- Finalize Task
  - Strong preference for SemEval2024 tasks!
  - Other task allowable with written permission (email)



NLP243 - Class Project

Teams and Tasks for the NLP243 Class Project (Fall 2023)

ianlane.us@gmail.com Switch account

* Indicates required question

Email *

Record ianlane.us@gmail.com as the email to be included with my response

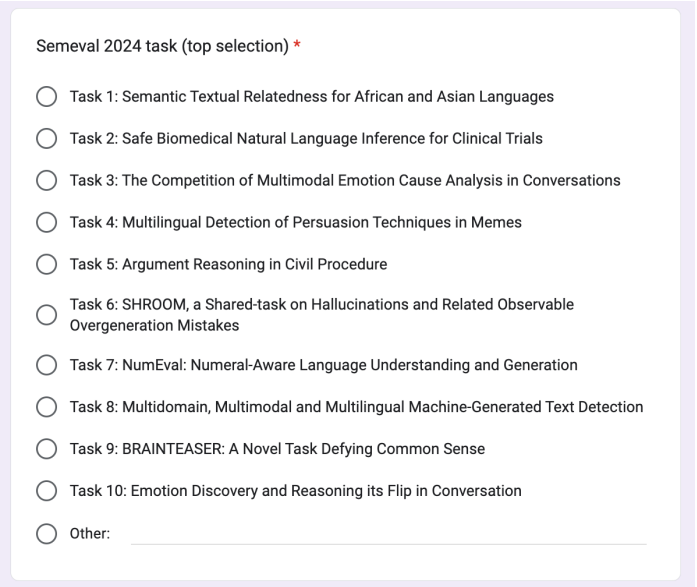
Full Name (UCSC email) *

Your answer

List of all team members: *

Full Name (UCSC email)
Full Name (UCSC email)
Full Name (UCSC email)

Your answer



Semeval 2024 task (top selection) *

- Task 1: Semantic Textual Relatedness for African and Asian Languages
- Task 2: Safe Biomedical Natural Language Inference for Clinical Trials
- Task 3: The Competition of Multimodal Emotion Cause Analysis in Conversations
- Task 4: Multilingual Detection of Persuasion Techniques in Memes
- Task 5: Argument Reasoning in Civil Procedure
- Task 6: SHROOM, a Shared-task on Hallucinations and Related Observable Overgeneration Mistakes
- Task 7: NumEval: Numeral-Aware Language Understanding and Generation
- Task 8: Multidomain, Multimodal and Multilingual Machine-Generated Text Detection
- Task 9: BRAINTEASER: A Novel Task Defying Common Sense
- Task 10: Emotion Discovery and Reasoning its Flip in Conversation
- Other: